

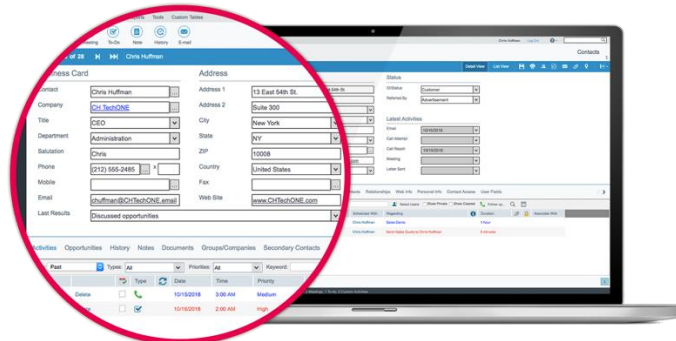
CS2300: File Structures and Introduction to Database Systems

Lecture 9: SQL

San Yeung

The Database System Environment

- A DBMS is a complex software system with many different components
- It can be divided into two parts
 - (a) the various user groups and their interfaces
 - (b) the internal modules of the DBMS responsible for storage of data and processing of transactions



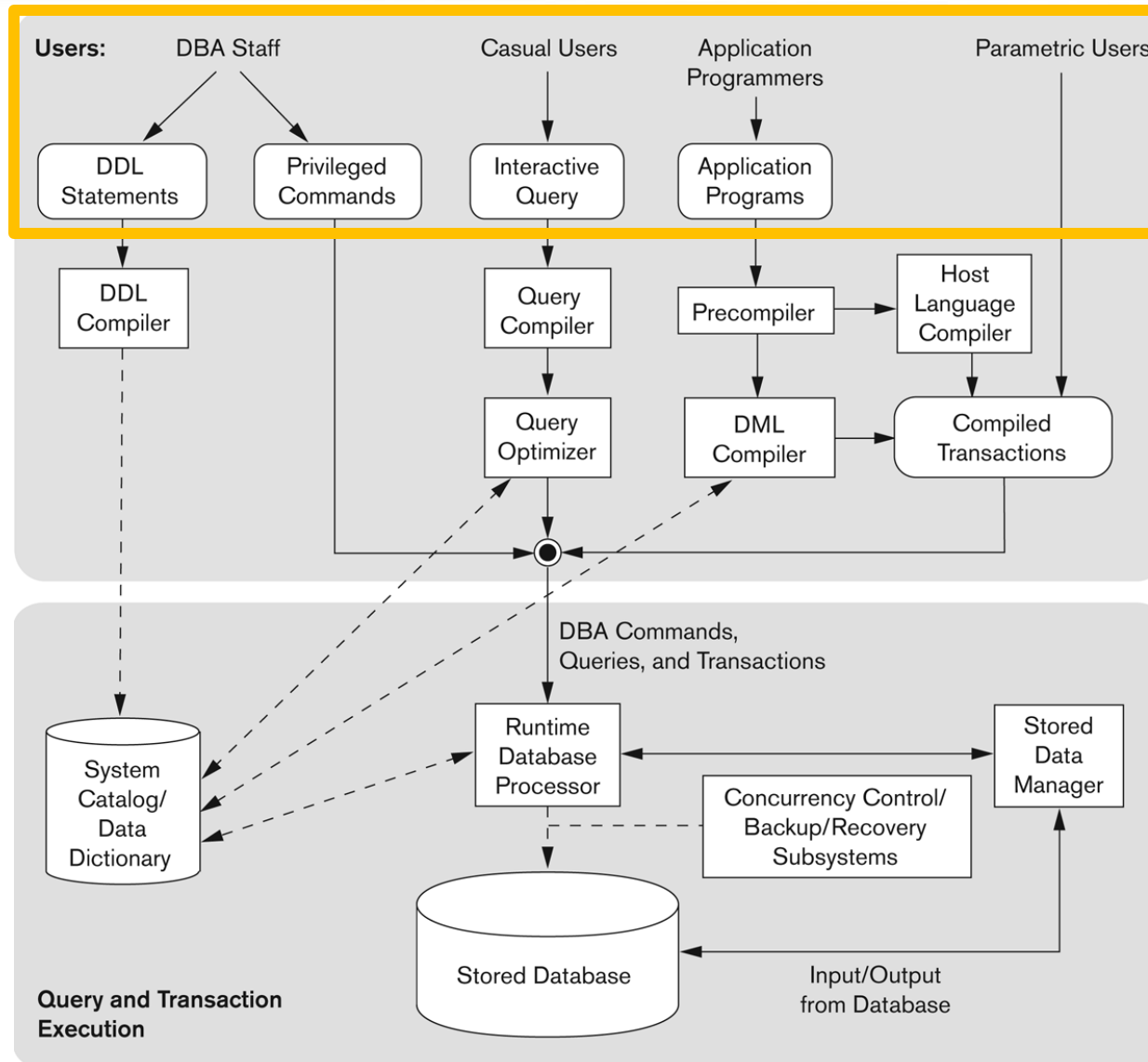


Figure 2.3

Component modules of a DBMS and their interactions.

The Database System Environment

DBA staff: works on defining and tuning the database using the **DDL** (*data definition language*) and other privileged commands

Causal users: work with interactive interfaces to formulate **queries**

Application programmers: create programs using some **host** programming languages (e.g. Java, C++, and Python)

- The DML compiler compiles data manipulation commands for database access

Parametric users: perform data entry work by supplying **parameters** to predefined transactions

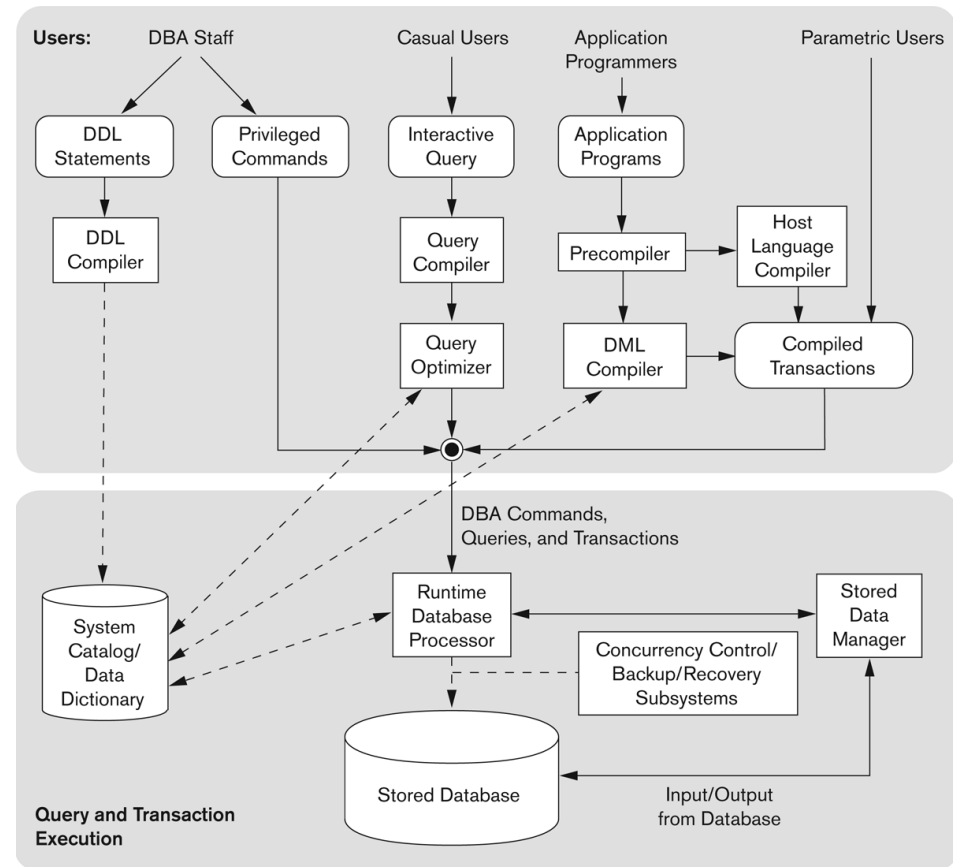


Figure 2.3
Component modules of a DBMS and their interactions.

DBMS Languages

- **DDL** (Data Definition Language)
 - Describe conceptual schema
- **SDL** (Storage Definition Language)
 - Specify internal schema
- **VDL** (View Definition Language)
 - Specify external view
- **DML** (Data Manipulation Language)
 - Manipulate the populated data
- Modern DB → All in one ☺ (E.g. **SQL**)

What is SQL?

The language has features for :

- Data definition and Data Manipulation,
- Transaction control (Transact-SQL, Ch. 20),
- Indexing (Ch.17),
- Security specification (Grant and Revoke- see Ch.30),
- Active databases (Ch.26),
 - an event-driven architecture which can respond to conditions both inside and outside the **database**
 - possible uses include *security monitoring, alerting, statistics gathering and authorization.*
- Multi-media (Ch.26),
- Distributed databases (Ch.23) etc.

SQL Standards

SQL

Paradigm	Multi-paradigm: declarative
Family	Programming language
Designed by	Donald D. Chamberlin Raymond F. Boyce
Developer	ISO/IEC
First appeared	1974; 45 years ago
Typing discipline	Static, strong
OS	Cross-platform
File formats	File format details
Filename extension	<code>.sql</code>
Internet media type	<code>application/sql</code> ^{[1][2]}
Developed by	ISO/IEC
Initial release	1986
Latest release	SQL:2016 (December 2016; 2 years ago)
Type of format	Database
Standard	ISO/IEC 9075
Open format?	Yes

Major implementations

Many

Dialects

SQL-86 · SQL-89 · SQL-92 · SQL:1999 ·
SQL:2003 · SQL:2006 · SQL:2008 · SQL:2011
· SQL:2016

Influenced by

Datalog

Influenced

CQL, LINQ, SPARQL, SOQL,
PowerShell,^[3] JPQL, JOOQ, N1QL

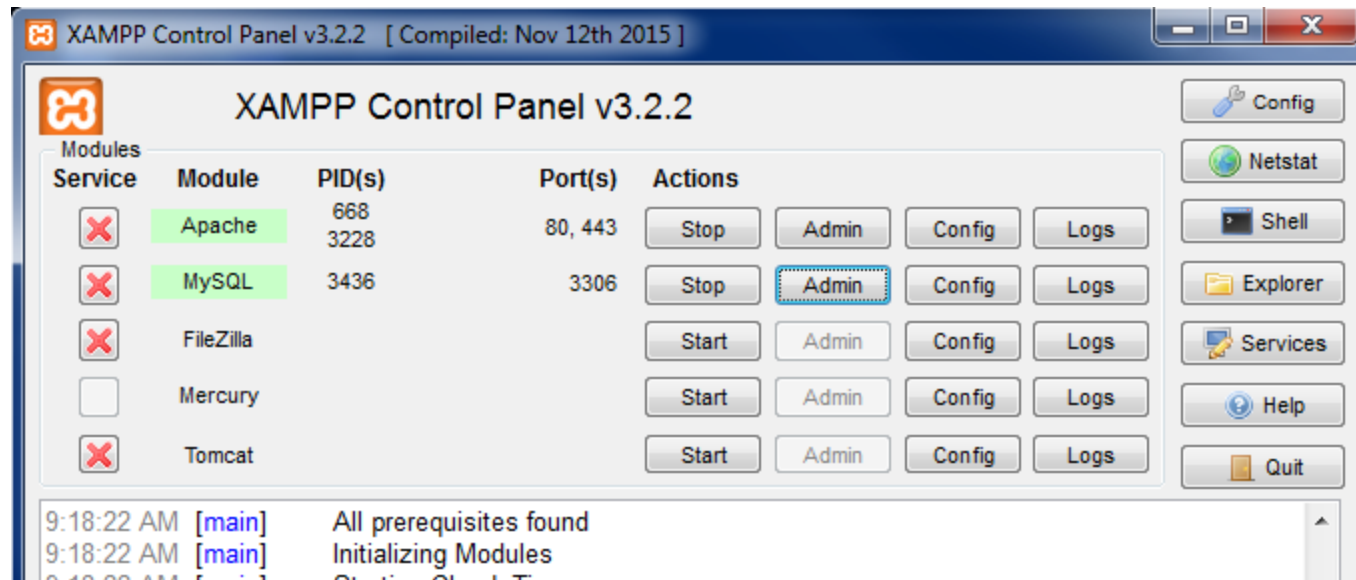


Structured Query Language at Wikibooks

Most Recent (ISO/IEC 9075: 2023)

Today's Outline

- Create/drop tables
- Insert/delete/update data in tables



CREATE TABLE Command

The *basic* format of the CREATE TABLE command is:

```
CREATE TABLE TableName(  
    Column1 DataType1 ColConstraint,  
    ...  
    ColumnN DataTypeN ColConstraint,  
    TableConstraint1,  
    ...  
    TableConstraintM  
);
```

- Specifying a new relation
 - Provide name of table
 - Specify attributes, their types and initial constraints

Example

```
SQL> CREATE TABLE DEPARTMENT (  
    DNAME      VARCHAR(15)      NOT NULL,  
    DNUMBER    INT,  
    MGRSSN     CHAR(9)          NOT NULL,  
    MGRSTARTDATE DATE,  
    PRIMARY KEY (DNUMBER)  
);
```

- Don't forget the semicolon ;
- Each statement in SQL ends with a **semicolon**
- Can optionally specify schema:
 - CREATE TABLE COMPANY.EMPLOYEE ...

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

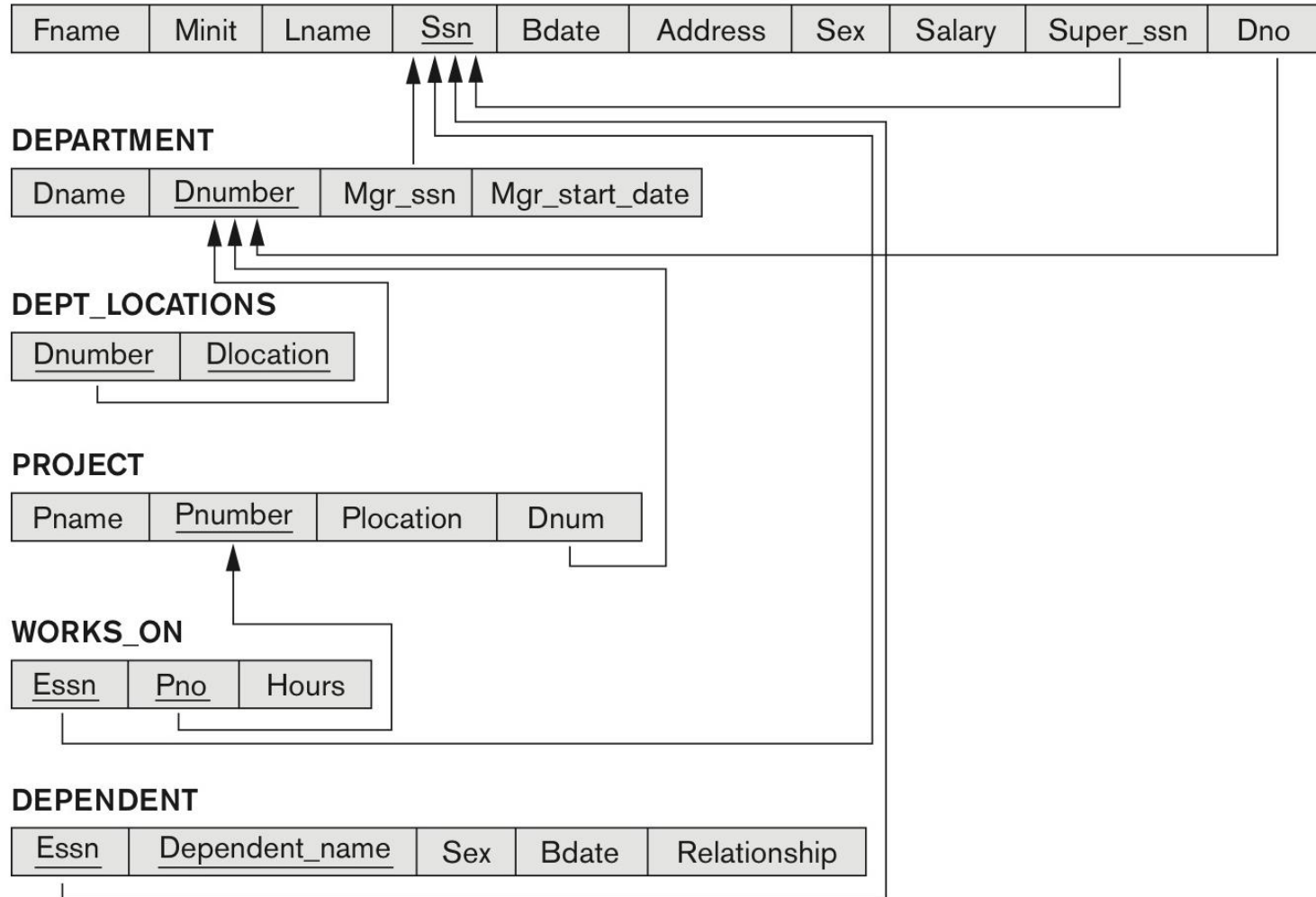
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



CREATE TABLE EMPLOYEE

(Fname	VARCHAR(15)	NOT NULL,
Minit	CHAR,	
Lname	VARCHAR(15)	NOT NULL,
Ssn	CHAR(9)	NOT NULL,
Bdate	DATE,	
Address	VARCHAR(30),	
Sex	CHAR,	
Salary	DECIMAL(10,2),	
Super_ssn	CHAR(9),	
Dno	INT	NOT NULL,

PRIMARY KEY (Ssn),

CREATE TABLE DEPARTMENT

(Dname	VARCHAR(15)	NOT NULL,
Dnumber	INT	NOT NULL,
Mgr_ssn	CHAR(9)	NOT NULL,
Mgr_start_date	DATE,	

PRIMARY KEY (Dnumber),

UNIQUE (Dname),

FOREIGN KEY (Mgr_ssn) **REFERENCES** EMPLOYEE(Ssn));

CREATE TABLE DEPT_LOCATIONS

(Dnumber	INT	NOT NULL,
Dlocation	VARCHAR(15)	NOT NULL,

PRIMARY KEY (Dnumber, Dlocation),

FOREIGN KEY (Dnumber) **REFERENCES** DEPARTMENT(Dnumber));

CREATE TABLE PROJECT

(Pname	VARCHAR(15)	NOT NULL,
Pnumber	INT	NOT NULL,
Plocation	VARCHAR(15),	
Dnum	INT	NOT NULL,

PRIMARY KEY (Pnumber),
UNIQUE (Pname),
FOREIGN KEY (Dnum) **REFERENCES** DEPARTMENT(Dnumber));

CREATE TABLE WORKS_ON

(Essn	CHAR(9)	NOT NULL,
Pno	INT	NOT NULL,
Hours	DECIMAL(3,1)	NOT NULL,

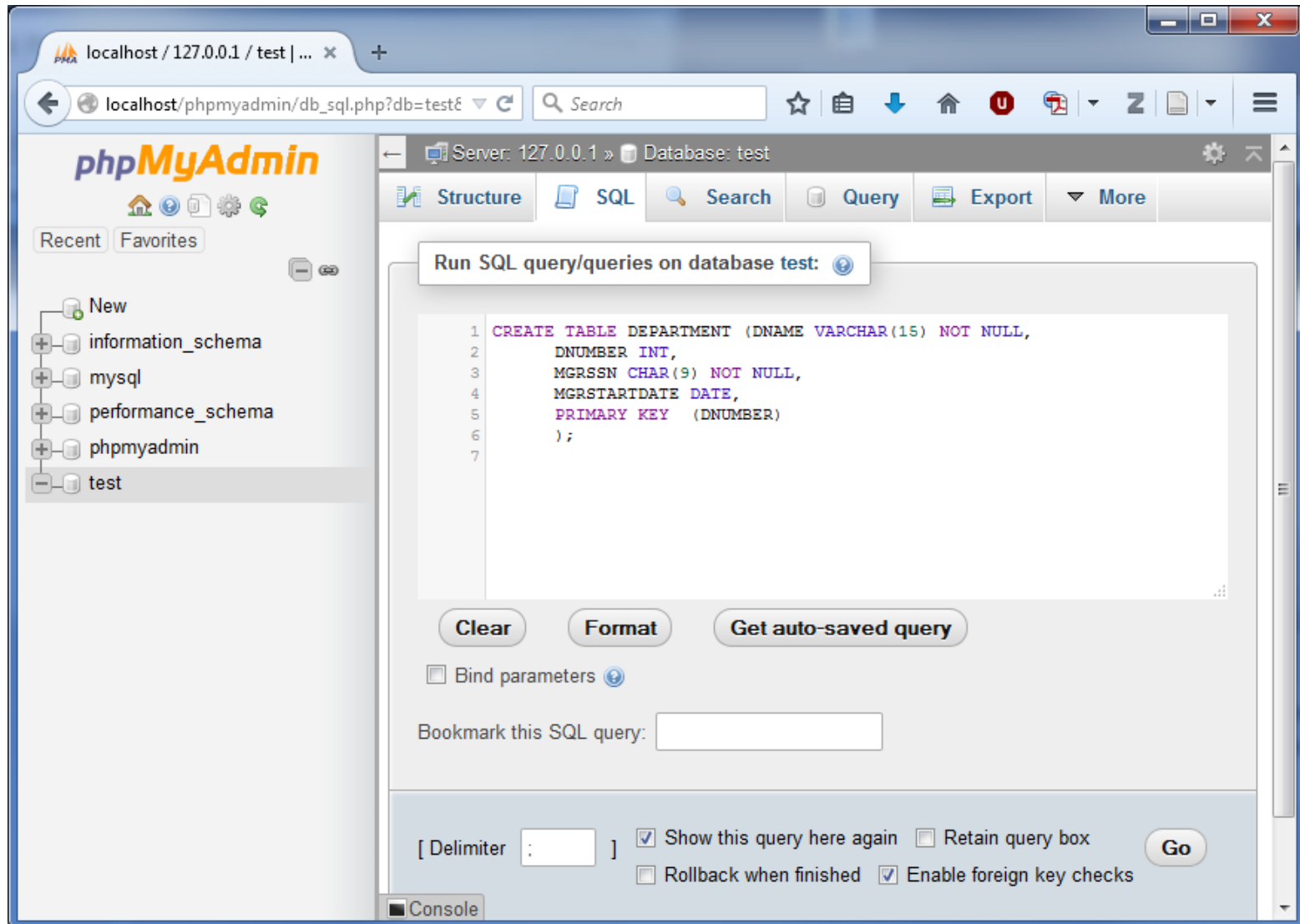
PRIMARY KEY (Essn, Pno),
FOREIGN KEY (Essn) **REFERENCES** EMPLOYEE(Ssn),
FOREIGN KEY (Pno) **REFERENCES** PROJECT(Pnumber));

CREATE TABLE DEPENDENT

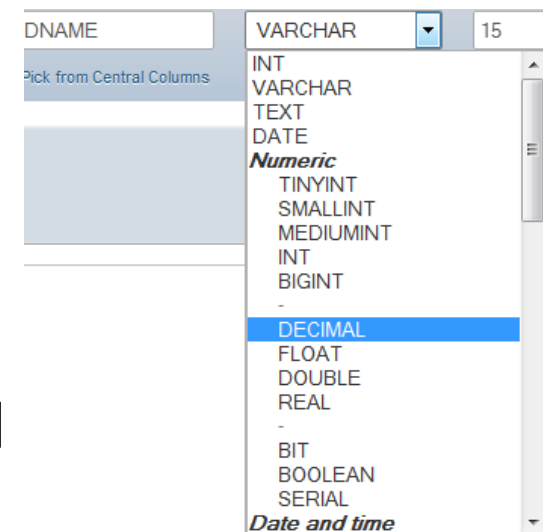
(Essn	CHAR(9)	NOT NULL,
Dependent_name	VARCHAR(15)	NOT NULL,
Sex	CHAR,	
Bdate	DATE,	
Relationship	VARCHAR(8),	

PRIMARY KEY (Essn, Dependent_name),
FOREIGN KEY (Essn) **REFERENCES** EMPLOYEE(Ssn));

Example



Attribute Data Types and Domains in SQL



- **Basic data types**

- **Numeric data types**

- Integer numbers: INTEGER, INT, and SMALLINT
 - Floating-point (real) numbers: FLOAT or REAL, and DOUBLE PRECISION

- **Character-string data types**

- Fixed length: CHAR(*n*), CHARACTER(*n*)
 - **Varying length**: VARCHAR(*n*), CHAR VARYING(*n*), CHARACTER VARYING(*n*)

Attribute Data Types and Domains in SQL (cont'd.)

– **Bit-string** data types

- Fixed length: `BIT (n)`
- Varying length: `BIT VARYING (n)`

– **Boolean** data type

- Values of `TRUE` or `FALSE` or `NULL`

– **DATE** data type

- Ten positions
- Components are `YEAR`, `MONTH`, and `DAY` in the form `YYYY-MM-DD`
- Multiple mapping functions available in RDBMSs to change date formats

Example: Datatypes in SQLite



Storage Classes and Datatypes

Each value stored in an SQLite database (or manipulated by the database engine) has one of the following storage classes:

- **NULL**. The value is a NULL value.
- **INTEGER**. The value is a signed integer, stored in 0, 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
- **REAL**. The value is a floating point value, stored as an 8-byte IEEE floating point number.
- **TEXT**. The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE).
- **BLOB**. The value is a blob of data, stored exactly as it was input.

Attribute Data Types and Domains in SQL (cont'd.)

- Additional data types
 - **Timestamp** data type
 - Includes the `DATE` and `TIME` fields
 - Optional `WITH TIME ZONE` qualifier
 - **INTERVAL** data type
 - Specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp
 - **DATE, TIME, Timestamp, INTERVAL** data types can be **cast** or converted to string formats for comparison.

Attribute Data Types and Domains in

• **Domain** SQL (cont'd.)

- Name used with the attribute specification
- Makes it easier to change the data type for a domain that is used by numerous attributes
- Improves schema readability
- Example:

- `CREATE DOMAIN SSN_TYPE AS CHAR(9);`

• **TYPE**

- User Defined Types (**UDTs**) are supported for object-oriented applications. (See Ch.12) Uses the command: `CREATE TYPE`

Constraints in Create Table

- Adding constraints to a table enables the database system to enforce data integrity.
- However, adding constraints also makes inserting data slower.
- Different types of constraints:
 - Not Null
 - Default Values
 - Unique
 - Primary Key
 - Foreign Key
 - Check Condition

Specifying Attribute Constraints

Other Restrictions on attribute domains:

- Default value of an attribute
 - **DEFAULT** <value>
- NULL is not permitted for a particular attribute (NOT NULL)
- **CHECK** clause
 - Dnumber INT NOT NULL CHECK
(Dnumber > 0 AND Dnumber < 21) ;

Not Null Constraint

Not Null = every tuple must have a real value for this attribute.

```
CREATE TABLE Employee(  
    ID          INTEGER NOT NULL,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1),  
    Salary      INTEGER NOT NULL,  
    Dept        INTEGER  
);
```

Default Values

Default value = a value to use whenever no other value of this attribute is known.

```
CREATE TABLE Employee(  
    ID          INTEGER NOT NULL,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1) DEFAULT('F'),  
    Salary      INTEGER NOT NULL,  
    Dept        INTEGER  
);
```

Specifying Key and Referential Integrity Constraints

- **PRIMARY KEY** clause
 - Specifies one or more attributes that make up the primary key of a relation
 - `Dnumber INT PRIMARY KEY;`
- **UNIQUE** clause
 - Specifies alternate (secondary) keys (called CANDIDATE keys in the relational model).
 - `Dname VARCHAR(15) UNIQUE;`

Declaring Keys

- Each table can have only one **Primary** key, but many **UNIQUE**'s.
- SQL does not allow NULL in primary key, but allows them in “unique” columns (which may have two or more nulls, but not repeated non-null values).

Unique Constraint (Syntax 1)

```
CREATE TABLE Employee(  
    ID          INTEGER UNIQUE NOT NULL,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1) default('F'),  
    Salary      INTEGER NOT NULL,  
    Dept        INTEGER  
);
```

Unique Constraint (Syntax 2)

```
CREATE TABLE Employee(  
    ID          INTEGER NOT NULL,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1) default('F'),  
    Salary      INTEGER NOT NULL,  
    Dept        INTEGER,  
    UNIQUE(ID)  
);
```

Question – Are they the same?

```
CREATE TABLE Sells (  
    bar CHAR(20),  
    beer VARCHAR(20),  
    price REAL,  
    UNIQUE (bar,beer)  
);
```

```
CREATE TABLE Sells (  
    bar CHAR(20) UNIQUE,  
    beer VARCHAR(20) UNIQUE,  
    price REAL  
);
```

Primary Key Constraint (Syntax 1)

```
CREATE TABLE Employee(  
    ID          INTEGER PRIMARY KEY,  
    Fname      VARCHAR(20),  
    Lname      VARCHAR(20),  
    Gender     CHAR(1),  
    Salary     INTEGER NOT NULL,  
    Dept       INTEGER,  
    UNIQUE (FNAME,LNAME)  
);
```

Primary Key implies: NOT NULL and UNIQUE.

Primary Key Constraint (Syntax 2)

Suppose that the primary key is (Fname, Lname).

```
CREATE TABLE Employee(  
    ID          INTEGER,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1),  
    Salary      INTEGER NOT NULL,  
    Dept        INTEGER,  
    PRIMARY KEY(Fname,Lname)  
);
```

Specifying Key and Referential Integrity Constraints (cont'd.)

- **FOREIGN KEY** clause
 - Default operation: reject update on violation
 - Attach **referential triggered action** clause
 - Options include SET NULL, CASCADE, and SET DEFAULT
 - Action taken by the DBMS for SET NULL or SET DEFAULT is the same for both ON DELETE and ON UPDATE
 - CASCADE option suitable for “relationship” relations

Foreign Key Constraint (Syntax 1)

```
CREATE TABLE Employee (  
  ID      INTEGER primary key,  
  Fname   VARCHAR(20),  
  Lname   VARCHAR(20),  
  Gender  CHAR(1),  
  Salary  INTEGER NOT NULL,  
  Dept    INTEGER REFERENCES Department(DeptNumber)  
);
```

Note 1: DeptNumber must be unique (or primary key) in Department

Note 2: The referencing attribute (Dept) can be null

Foreign Key Constraint (Syntax 2)

```
CREATE TABLE Employee (  
  ID          INTEGER primary key,  
  Fname      VARCHAR(20),  
  Lname      VARCHAR(20),  
  Gender     CHAR(1),  
  Salary     INTEGER NOT NULL,  
  DeptNumber  INTEGER REFERENCES Department  
);
```

Note: Don't need to specify the attribute in the referenced table if the name of the attributes in both tables are identical

Foreign Key Constraint (Syntax 3)

```
CREATE TABLE Employee(  
    ID          INTEGER primary key,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1),  
    Salary      INTEGER NOT NULL,  
    DeptNumber   INTEGER,  
    FOREIGN KEY (DeptNumber) REFERENCES Department  
);
```

This syntax is useful when a foreign key contains more than one attribute.

What Happens When a Foreign Key Constraint is Violated?

1. Insert or update a **Employee** tuple that refers to a nonexistent **Dept** in the **Department** table.

Always rejected

2. Delete or update **Dept** in the **Department** table that some **Employee** tuples refer to.
 - *Reject* by default
 - *CASCADE*
 - *SET NULL* or *SET DEFAULT*

Example

```
CREATE TABLE Employee(  
    ID          INTEGER primary key,  
    ... ..  
    DeptNumber  INTEGER,  
    FOREIGN KEY (DeptNumber) REFERENCES Department  
        ON DELETE CASCADE  
        ON UPDATE SET DEFAULT(000)  
);
```

Case 1: If Dept 123 is deleted from Department table, all tuples in Employee table with DeptNumber 123 will be automatically deleted.

Case 2: If Dept 123 is updated to 100 in the Department table, the value of Dept in Employee tuples which is 123 will be automatically set to 000.

is there a correct answer?

Giving Names to Constraints

```
CREATE TABLE Employee (  
  ID      INTEGER,  
  Fname   VARCHAR(20),  
  Lname   VARCHAR(20),  
  Gender  CHAR(1),  
  Salary  INTEGER NOT NULL,  
  CONSTRAINT Emp_Primarykey  
    Primary key (ID)  
);
```

Note 1: names of the constraints in the same relation must be unique

Note 2: it is useful when the constraint needs to be dropped later (useful for later altering)

Specifying Constraints on Tuples Using CHECK

- Additional Constraints on individual tuples within a relation are also possible using CHECK
- CHECK clauses at the end of a CREATE TABLE statement
 - Apply to each tuple individually
 - CHECK (Dept_create_date <= Mgr_start_date);

can you have more than one CHECK clauses in the same table?

Attribute-Based Check

- Follow an attribute by a condition that must hold for that *attribute* in each tuple of its relation.
- Condition is **checked only when the associated attribute changes** (*i.e.*, an insert or update occurs).

```
CREATE TABLE Employee(  
    ID          INTEGER primary key,  
    Fname       VARCHAR(20),  
    Lname       VARCHAR(20),  
    Gender      CHAR(1) check(gender='F' or  
                        gender='M'),  
    Salary      INTEGER check(Salary>=0),  
    DeptNumber  INTEGER  
);
```

Tuple-Based Check

- Separate element of table declaration.
- Checked whenever a tuple is inserted or updated.

```
CREATE TABLE Sells (  
    bar CHAR(20),  
    beer CHAR(20),  
    price REAL,  
    PRIMARY KEY (bar,beer),  
    CHECK (bar = 'Joe's Bar' OR price <= 5.00)  
);
```

Only Joe's Bar can sell beer for more than \$5? why??

Delete a Table

- DROP – a scheme change command

- Basic Format:

DROP TABLE Table_name RESTRICT;

DROP TABLE Table_name CASCADE;

“RESTRICT”: drop the table only when it is not **referenced**

“CASCADE”: will drop the constraints and views that reference the table

- DROP TABLE command remove **all records** in the table and the **table definition**.
- To delete the table **DEPARTMENT**:
SQL> DROP TABLE DEPARTMENT CASCADE;

INSERT, DELETE, and UPDATE Statements in SQL

- Three commands used to modify the database (tables):
 - INSERT, DELETE, and UPDATE
- **INSERT** typically inserts a tuple (row) in a relation (table)
- **UPDATE** may update a number of tuples (rows) in a relation (table) that satisfy the condition
- **DELETE** may also update a number of tuples (rows) in a relation (table) that satisfy the condition

Inserting a Row

To insert a row into the Employee table

Employee(id:integer, fname:varchar(20), gender:varchar(1),
deptnumber:integer, salary:integer)

- **Approach 1:** The attributes don't have to be specified if values are specified for all columns and in the order defined by the table

```
SQL> Insert Into Employee  
      Values (122, 'Bob', 'M', 12, 110000);
```

Inserting a Row

To insert a row into the Employee table

Employee(**id**:integer,**fname**:varchar(20),**gender**:varchar(1),
deptnumber:integer, **salary**:integer)

- **Approach 2:** Only few of attributes are assigned values for the new tuple.

```
SQL> Insert Into Employee(id,deptnumber)  
      Values(123,13) ;
```

Inserting a Row

- Approach 3: use the SQL query to retrieve data from other tables and then insert them into the current table

```
SQL> Insert Into Employee(id,fname)  
      Select [attr1], [attr2]  
      From   [table_name]  
      Where  [condition];
```

Deleting Rows

- Removes tuples from a relation
- General format:

DELETE FROM *Table*

WHERE *Condition*;

- Deletes *all* rows satisfying ***Condition*** from ***Table***
- Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
- A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table

Example

- Remove the employee with id 121 from the Employee table:

```
DELETE FROM Employee WHERE id = 121;
```

- Remove all employees having a salary greater than 100,000:

```
DELETE FROM Employee WHERE Salary > 100000;
```

- Remove all records in the Employee table

```
DELETE FROM Employee;
```

Deleting Rows

- The *Condition* can also involve SQL queries.
- Example:

```
DELETE FROM Employee  
WHERE Deptnumber IN (Select Dnumber  
                     From Department  
                     Where Dname = `Research`);
```

The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

Updating Rows

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- General format:

UPDATE Table

SET Column1=value1,...,ColumnN=valueN

WHERE Condition

- Referential integrity specified as part of DDL specification is enforced

Example

1. Now we can reduce salaries instead of firing employees:

```
UPDATE Employee  
    SET Salary = 100000  
    WHERE Salary > 100000;
```

2. UPDATE PROJECT
 SET PLOCATION = 'Bellaire',
 DNUM = 5
 WHERE PNUMBER=10

what if no PNUMBER is = 10?

Updating Rows

- The **Condition** can also involve SQL queries.
- Example:

```
UPDATE Employee
SET      Salary = Salary*1.1
WHERE    Deptnumber IN (Select Dnumber
                        From Department
                        Where Dname = `Research`);
```

is a 10% raise enough for you?

Table Alteration



Altering Tables

- Table definitions can be altered after their creation
 - DBA may want to change the schema while the database is operational
 - Does not require recompilation of the database schema
- Use the reserved words **ALTER TABLE**
- Options:
 - Adding columns: **ADD COLUMN**
 - Changing columns' definition: **ALTER COLUMN**
 - Dropping columns: **DROP COLUMN**
 - Adding or dropping constraints: **ADD, DROP CONSTRAINT**
 - And more...

Altering Tables

- Adding a column:
ALTER TABLE Employee **ADD COLUMN** mname **VARCHAR(20);**
- Changing columns' definition:
ALTER TABLE Employee **Modify** mname **varchar(10);**
- Dropping columns:
ALTER TABLE Employee **DROP COLUMN** Mname **CASCADE;**
- Adding constraints:
ALTER TABLE Employee **ADD** **check(age>0);**



```
Oracle SQL*Plus
File Edit Search Options Help
12 ;
Table created.
SQL> ALTER TABLE EMPLOYEE ADD FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE (SSN);
Table altered.
SQL> |
```

Dropping Columns, Default Values

- To drop a column
 - Choose either `CASCADE` or `RESTRICT`
 - `CASCADE` would drop the column from views etc.
`RESTRICT` is possible if no views refer to it.

```
ALTER TABLE COMPANY.EMPLOYEE  
DROP COLUMN Address CASCADE;
```

- Default values can be dropped and altered :

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER COLUMN Mgr_ssn DROP DEFAULT;
```

```
ALTER TABLE COMPANY.DEPARTMENT  
ALTER COLUMN Mgr_ssn SET DEFAULT '333445555';
```

Table 7.2 Summary of SQL Syntax

Table 7.2 Summary of SQL Syntax

```
CREATE TABLE <table name> ( <column name> <column type> [ <attribute constraint> ]
                             { , <column name> <column type> [ <attribute constraint> ] }
                             [ <table constraint> { , <table constraint> } ] )
```

```
DROP TABLE <table name>
```

```
ALTER TABLE <table name> ADD <column name> <column type>
```

```
SELECT [ DISTINCT ] <attribute list>
```

```
FROM ( <table name> { <alias> } | <joined table> ) { , ( <table name> { <alias> } | <joined table> ) }
[ WHERE <condition> ]
```

```
[ GROUP BY <grouping attributes> [ HAVING <group selection condition> ] ]
```

```
[ ORDER BY <column name> [ <order> ] { , <column name> [ <order> ] } ]
```

```
<attribute list> ::= ( * | ( <column name> | <function> ( ( [ DISTINCT ] <column name> | * ) ) )
                      { , ( <column name> | <function> ( ( [ DISTINCT ] <column name> | * ) ) ) } ) )
```

```
<grouping attributes> ::= <column name> { , <column name> }
```

```
<order> ::= ( ASC | DESC )
```

```
INSERT INTO <table name> [ ( <column name> { , <column name> } ) ]
```

```
( VALUES ( <constant value> , { <constant value> } ) { , ( <constant value> { , <constant value> } ) } )
| <select statement> )
```

Table 7.2 Summary of SQL Syntax

DELETE FROM <table name>

[WHERE <selection condition>]

UPDATE <table name>

SET <column name> = <value expression> { , <column name> = <value expression> }

[WHERE <selection condition>]

CREATE [UNIQUE] INDEX <index name>

ON <table name> (<column name> [<order>] { , <column name> [<order>] })

[CLUSTER]

DROP INDEX <index name>

CREATE VIEW <view name> [(<column name> { , <column name> })]

AS <select statement>

DROP VIEW <view name>

NOTE: The commands for creating and dropping indexes are not part of standard SQL.

Summary

- SQL
 - A Comprehensive language for relational database management
 - Data definition, queries, updates, constraint specification, and view definition
- Covered :
 - Data definition commands for creating tables
 - Commands for constraint specification
 - Simple retrieval queries
 - Database update commands